

Лабораторная работа №8

**Реализация основных моделей в дискретно-событийном
подходе**

Андрей Геннадьевич Софич

Содержание

1	Цель работы	4
2	Задание	5
3	Выполнение лабораторной работы	6
4	Выводы	14

Список иллюстраций

3.1	Инициализация проекта	6
3.2	Подключение необходимых пакетов	6
3.3	Создание файла для реализации SIR	7
3.4	Скрипт симуляции	7
3.5	График	8
3.6	Код в стиле jupyter notebook	8
3.7	Замена файла	8
3.8	Сравнение графиков	9
3.9	Добавление таблицы	9
3.10	Таблица с данными SIR	10
3.11	Код в стиле jupyter notebook	10

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR.

2 Задание

Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

3 Выполнение лабораторной работы

Создаем рабочий каталог и инициализируем проект в julia (рис. 3.1).

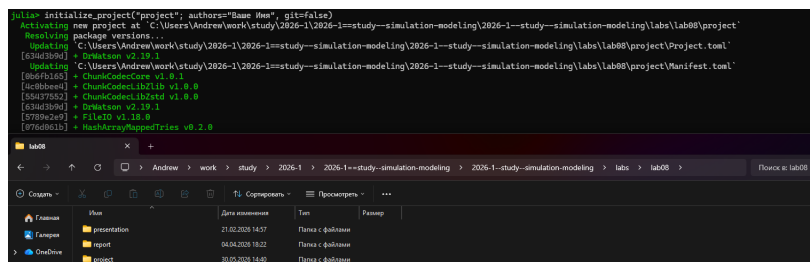


Рисунок 3.1: Инициализация проекта

Создаем файл, где прописываем пакеты, необходимые для выполнения работы и запускаем его (рис. 3.2).

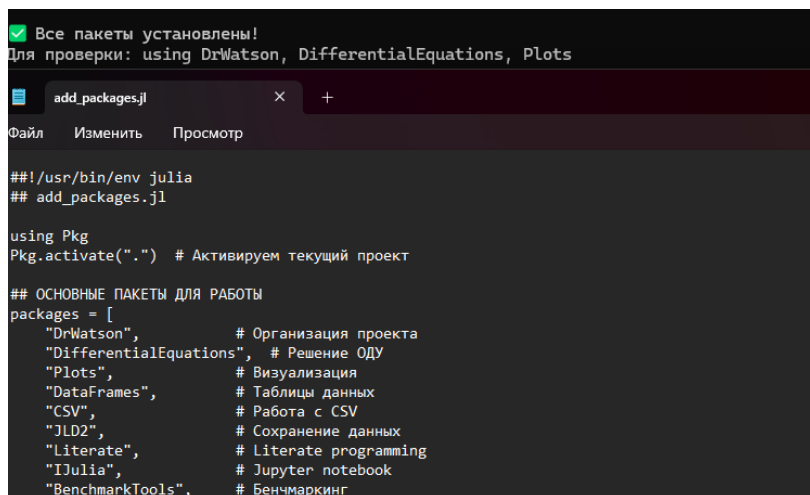
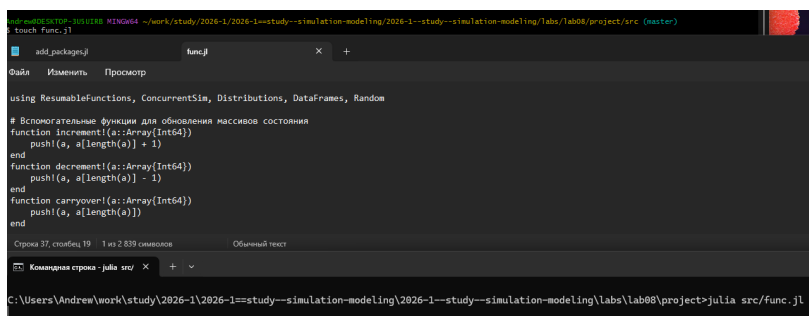


Рисунок 3.2: Подключение необходимых пакетов

Создаем файл в папке src, реализуем в нем модель SIR, определяем структуру

данных, входные данные, прописываем функции мутации массивов статистики (рис. 3.3).



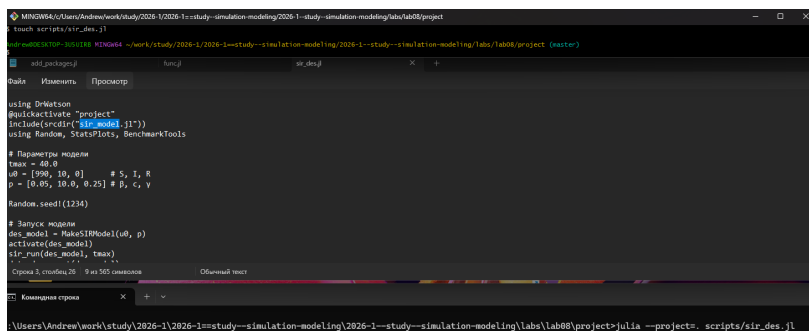
```
touch func.jl

using ResumableFunctions, ConcurrentSim, Distributions, DataFrames, Random

# Вспомогательные функции для обновления массивов состояния
function increment!(a::Array{Int64})
    push!(a, a[length(a)] + 1)
end
function decrement!(a::Array{Int64})
    push!(a, a[length(a)] - 1)
end
function carryover!(a::Array{Int64})
    push!(a, a[length(a)])
end
```

Рисунок 3.3: Создание файла для реализации SIR

Прописываем и запускаем скрипт запуска, прописываем начальные параметры и запускаем симуляцию (рис. 3.4).



```
using Distributions
function make_model!(p)
    include(srcdir("sir_model.jl"))
    using Random, StatsPlots, BenchmarkTools

    # Параметры модели
    tmax = 40.0
    u0 = [990, 10, 0] # S, I, R
    p = [0.05, 10.0, 0.25] # β, c, γ
    Random.seed!(1234)

    # Запуск модели
    des_model = MakeSIRModel(u0, p)
    sir_run(des_model, tmax)
end
```

Рисунок 3.4: Скрипт симуляции

Получаем результаты, на которых отлично видно структуру поведения агентов: восприимчивый-зараженный-выздоровевший (рис. 3.5).

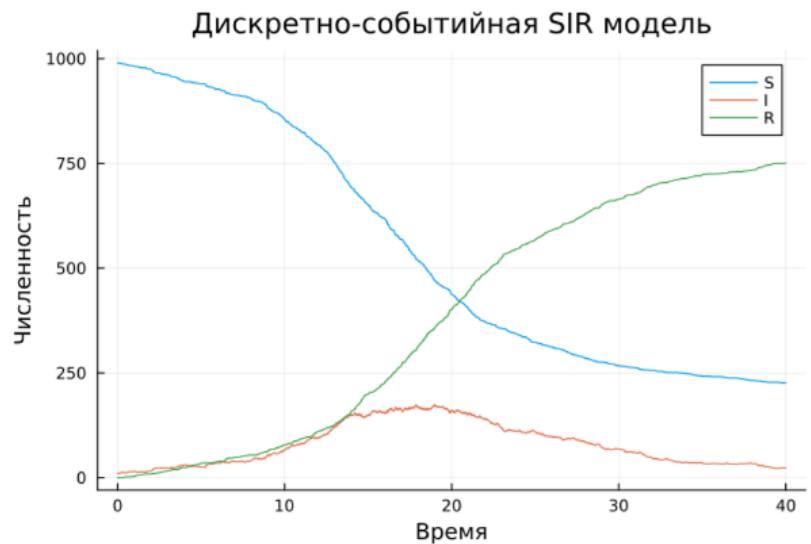


Рисунок 3.5: График

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.6).

```

Quarto: C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab00\project\notebooks\src_des\src_des.qmd
Info: generating notebook from C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab00\project\notebooks\src_des.jl
Info: writing result to C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab00\project\notebooks\src_des\src_des.ipynb
Notebook: C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab00\project\notebooks\src_des\src_des.ipynb

[Quarto] Все файлы созданы.

src_des.ipynb X
C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab00\project\notebooks\src_des> src_des.ipynb using DataFrames
% Generate + Code + Markdown | Run All | Clear All Outputs | Offline

>
using DataFrames
@docactivate "project"
include("src_model.jl")
using Random, StatsPlots, BenchmarkTools

Параметры модели

tau = 40.0
u0 = [100, 10, 0] # S, I, R
p = [0.0, 10.0, 0.25] # b, c, r
Random.seed!(1234)

Запуск модели

```

Рисунок 3.6: Код в стиле jupyter notebook

Заменяем экспоненциальное время выздоровления на фиксированную величину $1/\gamma$ в файле src (рис. 3.7).

```

C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab00\project>julia src\src_model.jl

src_model.jl
# Основная логика жизни индивида
@resumable function live(env::ConcurrentSim.Simulation, individual::SIRPerson, m::SIRModel)
    while individual.status == :S
        @yield timeout(env, 1/m.y)
    end
end

```

Рисунок 3.7: Замена файла

Запускаем файл симуляции и смотрим на результаты до и после, отсутствуют хвосты распределения и выздоровевшее стало более синхронным (рис. 3.8).

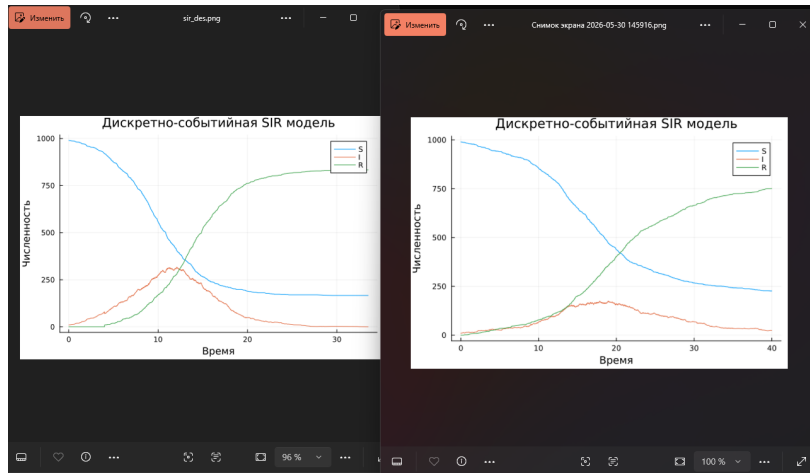


Рисунок 3.8: Сравнение графиков

Добавляем возможность получать статистические данные в виде таблицы (рис. 3.9).

```
C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1==study--simulation-modeling\labs\lab08\project>julia scripts\sir_des.jl
Activating project at "C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1==study--simulation-modeling\labs\lab08\project"
C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1==study--simulation-modeling\labs\lab08\project>
julia> using Plots
julia> ylab = "Численность";
julia> title = "Дискретно-событийная SIR модель";
julia>
julia> savefig(plotdir("sir_des.png"))
julia> filename = "sir_${id[1]}_${id[2]}_${id[3]}_${id[4]}_${id[5]}.csv"
julia> CSV.write(datadir("sims", filename), data_des)
```

Рисунок 3.9: Добавление таблицы

Проверяем таблицу с данными (рис. 3.10).

[illegible]

Рисунок 3.10: Таблица с данными SIR

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.11).

[illegible]

Рисунок 3.11: Код в стиле jupyter notebook

```
using DrWatson
@quickactivate "project"
```

```
include(sourcedir("sir_model.jl"))
using Random, StatsPlots, BenchmarkTools
```

Параметры модели

```
tmax = 40.0
u0 = [990, 10, 0]      # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 

Random.seed!(1234)
```

Запуск модели

```
des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)
```

Визуализация

```
@df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlab = "time",
    ylab = "population",
    title = "SIR model",
)
savefig(plotsdir("sir_des.png"))
```

```

using DrWatson
@quickactivate "project"
include(sourcedir("sir_model.jl"))
using Random, StatsPlots, BenchmarkTools
using CSV, Dates

```

Параметры модели

```

tmax = 40.0
u0 = [990, 10, 0]      # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 

Random.seed!(1234)

```

Запуск модели

```

des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)

```

Визуализация

```

@df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlab = "time",
    ylab = "count",
    title = "SIR model",
)

```

```
)  
savefig(plotsdir("sir_des.png"))  
filename = "sir_$(u0[1])_$(u0[2])_$(p[1])_$(p[2])_$(p[3]).csv"  
CSV.write(datadir("sims", filename), data_des)
```

4 Выводы

В данной работе мы изучили и применили дискретно-событийное моделирование.